Homework #2

Shayan Karmaly
skarmaly3@gatech.edu

QUESTION 1

1.1

A task that has become an invisible by learning is typing on a keyboard.

1.2

The interface consists of a keyboard, the trackpad and the computer screen. The underlying objects consist of the characters, word editor and cursor. The components of the interface I used to think about were the individual characters that make up the words I was trying to piece together via a group of characters. Also how to best align my hand positioning to be in reach of the keys I wanted to press; I would always be searching for the two little grooves around the *F* and the *J* keys, but now my hands gravitate their automatically once my hands land on the keyboard. I would always be struggling to get back into typing rhythm going back and forth between the keyboard and the trackpad. My thought process now is automatic in the sense that I'm not having to search around for where certain keys are for characters I'm trying to type. In fact, my thought process involves *chunking* each sequence of characters I type into words compared to individual characters like before. I also use hot keys such as *Ctrl-z* for undo and *Ctrl-C* + *Ctrl-X* for copying and cutting. I believe the interface became so invisible because it allowed me to learn and combine multiple modalities together using visual, audible, and verbal cues that reduce cognitive load. I can see the text visually and verbally on the screen while I hear myself type on the keyboard. Seeing this happen in real time, the multiple modalities complement each other allowing for less time that I must think about using the interface to complete the task and more time for me to complete the task. Another factor to consider is muscle memory in which the physical act of repeating something repeatedly has allowed me to get better at it as a skill. I might redesign to get to the point of invisibility more quickly by making hot keys and keyboard functionalities more discoverable than they currently are. While on a Mac, it does show me the hot keys next to the commands when I hover over them in the control menu, certain shortcuts

like *cmd + tab* or swiping the trackpad left or right to toggle between applications are not readily discoverable. I'd redesign the interface to have visual, on-screen hints that display hot keys for actions that users perform often to help them discover it and use the interface more efficiently to reach the point of invisibility.

While walking on the street, the interface can be designed to overcome constraints by limiting controls of the app to what are necessary such as auto pausing when a person stops walking because their attention is briefly directed to their physical environment and not on their phone anymore. Cognitive and physically demanding features like a '*comment*' can be temporarily disabled. Haptic feedback at the end of a video to signal it done in case a user is looking away from the video but still ingesting the content audibly is another alternative. These changes can assist in allowing the user to use the interface while in the context of walking which might reduce their physical senses and cognitive focus on the activity of scrolling.

While sitting on the couch, the interface can be designed to overcome constraints to have richer interactions because the interface knows the user has most of their cognitive power on the activity of scrolling at hand. For this, actions like tapping to expand features on comments/threads and ensuring all controls are fully enabled. The design could also limit notifications or unnecessary interruptions like notifications of messages from other people, being tagged on a post, comment, or thread so as not to distract the user from the activity of scrolling.

While eating at the dinner table, the interface can be designed to by optimizing the actions users do to achieve their goals that would need usually need two hands into one. Actions like typing a comment, searching for a video or topic, navigating the menus, could be designed for single swipe-based controls or voice input where users can speak commands like *'rotate my screen'* or *'zoom in on this comment'*. Similar to the walking context, clearer cues for when a video is done can be implemented like haptic feedback in the case that a user is looking down to grab some food or away briefly for any reason.

QUESTION 2

2.1

While ***using a nutrition-tracking application to track your nutritional needs***, I mostly experience visual feedback compared to auditory or haptic. The app I use is called *Cronometer* and it's visually designed with the tabs at the bottom of the screen to toggle between different categories to act in. The main tab I use is primarily the *Diary* tab which consists of a daily input of total food. It displays a user's targets [1]and how close or over they are on hitting those with a visual progress bar through color coding. In each meal, I'm able to log my food entries inferred by the + sign which displays a sub-menu of what I'd like to log, as well as a database of foods that I can query results from. Haptic feedback occurs when switching tabs such as from the *diary* to *foods* tab or when I press a button such as *add to diary* when adding food that confirms my action has been received by the system. Auditory feedback occurs when I'm typing text on the search bar, complemented by the visual feedback of seeing those letters, letting me know that what I'm typing is being translated to the system.

2.2

A visual perception that could be used to give feedback is for the system to auto-generate suggested foods to log in a grayed-out text in the search box when a user goes to type in some food to add for a specific meal. This is a suggestion for food they might be logging based on their user data. For example, as a body-builder I typically eat the same things every day and the system can recognize that for *Breakfast*, visually display some suggested foods that I will most likely want to log that I ate such as *Eggs*. A haptic perception that could be used to give feedback to a user that they've either exceeded or are below their daily targets. A double vibration can be used to signal that they are under their daily targets, and a triple vibration can be used to signal that they have exceeded it. This can also be complemented visually with a notification pop up saying whether they're under or exceeded their calorie targets. An audible perception that could be used to give feedback is a unique auditory chime when successfully logging a food that would signal the confirmation of logging the food to the user.

[1] Macronutrients and micronutrients of a food.

2.3

A different kind of human perception would be using the naturally producing hunger hormone, *ghrelin*, to be used by the app to then produce some feedback to the user that it's time to log some food. There are two cases in which a user has eaten or not; if ghrelin is high, we know the user hasn't eaten and if it's low, they've most likely eaten. In either case, we can use this perception to provide feedback to them and remind them to *log what you're about to eat!* Or, ask them to *log what food you ate!* These reminders can be visually as a notification on their phone from the app, audibly mimicking a stomach growl, or haptic feedback like multiple vibration pulses.

Figure 1 – My car's infotainment system with physical buttons outside of the digital touchscreen with digital buttons as well.

QUESTION 3

3.1 Emphasizing essential content while minimizing clutter

The infotainment system on my car is a great example of an interface that violates this tip. As seen on Figure 1, there is a left-hand column on the screen containing many shortcut buttons a user might use. In the aim to assist a user's experience and might be useful in certain situations, it fails to consider the users cognitive load while driving. These are non-essential functions and provides unnecessary information to the user. I also wanted to note the redundancy across the physical and digital buttons of Phone, Nav, and Audio/Video specifically while driving. While redundancy of essential functions doesn't violate this tip, a user's cognitive load might increase because of the redundancy while driving making it difficult for users to decide between input methods instead of consolidating it to just one.

My redesign of this would be to consolidate the left-hand column controls into their respective categories. A single Settings icon that contains the In control apps and Extra features buttons, put Phonebook when a user clicks the Phone button, put Audio settings when a user clicks the Audio/Video button, and put Take me home when a user presses the Nav button. This will allow the user to still have those functions but clean up the base user interface that they see while driving. Additionally, the system would detect if a user were driving and not display Phone, Audio/Video and Nav and instead have the user press one of the physical buttons that would take them their desired screen.

3.2 Using multiple modalities

An interface from my everyday life that violates this tip is my pod-based espresso machine. The machine has a latch at the top to insert a pod, a button at the bottom to turn it on or off, a button in the middle with a + and – at the top and bottom of that button respectively, as well as a digital coffee icon at the top once you turn it on. The violation of this tip occurs because the different modalities being used, visual, and haptic, do not complement each other to guide them in making an espresso shot. When you turn the machine on, it doesn't communicate if a pod should be inserted, the coffee icon isn't enough to indicate whether the machine is ready or waiting for the user to input a pod. When I do happen to insert a pod, nothing changes. As a novice, I didn't know what the + and –

symbols meant when creating an espresso shot, I could assume that it was strength due to prior knowledge, but it could've also been volume or something else.

My redesign of this would be to have the multiple modalities of visual, auditory, and haptic complement each other by using them together in each step of the espresso making process. When first turning the machine on, users see the latch light up in a bright *red* or *green* to indicate whether the latch needs a pod or not to make the espresso. Upon closing the latch, a successful audio chime will be emitted to signify that the pod was accepted. Instead of choosing a button for strength, I'd redesign the machine to have a type of custom dial that visually signifies a stronger shot at the top and the more you move it to the bottom, the weaker it will be. I'd also redesign to have a visual, pulsing light on the button to make the espresso shot to signify that it's ready to brew as well as a simple text like *START* on it.

QUESTION 4

4.1

An interface that is intolerant of errors the user commits is driving my manual car.

The interface includes multiple pieces that work together including the clutch, brake pedal, gas pedal, parking brake, dashboard, gear shifter, as well as the multiple mirrors and the windshield assist in the process of driving of a manual car. The user's error of driving while the parking brake is still engaged is responded by this red *brake* sign on the small dashboard that sits behind the steering wheel. Audibly, the user hears the car's engine getting louder but physically isn't moving. Other than this, the user has to recall that the parking brake is still engaged and infer what the error is, otherwise the car won't move. The error is very easy to commit because most users, experienced and novice, will leave the parking brake engaged so the car doesn't roll when they leave the car so when they come back to drive it, they might forget that they still have it engaged. The penalty associated with it is potentially ruining the clutch, burning through gas, ruining the parking brake because the user is attempting to drive the car without the right settings of the parking brake disengaged.

4.2

Constraints such as not allowing the user to release the clutch past the biting point while the clutch is engaged to avoid this error. This would still allow necessary actions like hill starts to be done while still having the parking brake engaged. As the parking brake loosens, the clutch release also loosens. This would prevent the user error from occurring *until* the parking brake is disengaged.

Improved mappings like a warning sign displayed on the dashboard of someone's hand disengaging the parking brake before they start driving, similar to when someone's seatbelt is off and they start driving, as well as a distinct chime before driving can be implemented to avoid this error.

Improved affordances to ensure a user doesn't try to drive the car before disengaging the parking brake would be to design the interface so that the parking brake invites the user to disengage the brake before driving. Improvements like matching the color of the button at the end of the parking brake that gets pushed to the same color as the gear shifter or directly aligning the parking brake behind the gear shifter can be physically altered to signify their relationship.

REFERENCES